

You are refactoring the codebase.

Your task is to refactor `admin.js` file into a clean modular structure while preserving **100% existing behavior**.

This is a **strict refactor only**.

Primary Objectives

1. Split the giant file into focused modules with clear responsibilities.
2. Reduce duplication and improve maintainability.
3. Preserve exact behavior and functionality.

You MUST NOT:

- Add new features
- Change UX behavior
- Change business logic
- Change API behavior
- Change database contracts
- Change naming of DOM ids/classes unless required for imports
- Rewrite logic unnecessarily
- Introduce frameworks
- Convert to TypeScript
- Replace existing API abstractions (`Admin` , `Invites` , `Services` , `ServiceRequests` , `Organizer` , `apiFetch`) unless required for imports

This refactor must be **safe and surgical**.

Critical Requirement

This refactor affects code that communicates with `admin.js`.

You MUST scan and update the ENTIRE codebase for dependencies that currently reference functionality from the original giant file.

This includes:

- HTML files
- Inline event handlers
- Imported scripts
- Other JS files
- Script paths
- DOMContentLoaded bindings

- Any `onclick`, `onchange`, `onsubmit`, `window.functionName`, or global function access

You MUST update all paths and references so the system still works after the split.

No broken imports.

No orphaned functions.

No unresolved references.

No console errors.

No missing script paths.

If inline HTML still depends on global scope access, expose only the minimum required functions intentionally.

Required Folder Structure

Refactor into this structure exactly:

```
js/
├── admin/
│   ├── admin.js
│   ├── state/
│   │   └── adminState.js
│   ├── loaders/
│   │   └── dataLoaders.js
│   ├── navigation/
│   │   ├── viewManager.js
│   │   └── sidebar.js
│   ├── overview/
│   │   ├── overviewRenderer.js
│   │   └── activityFeed.js
│   └── users/
│       ├── userFilters.js
│       ├── userRenderer.js
│       └── userActions.js
```

```
├── userDetails.js
├── requests/
│   ├── requestRenderer.js
│   └── requestReview.js
├── families/
│   ├── familyRenderer.js
│   ├── familyModal.js
│   ├── familyMembers.js
│   └── addMemberView.js
├── subscriptions/
│   └── subscriptionRenderer.js
├── invites/
│   ├── inviteRenderer.js
│   └── inviteModal.js
├── promos/
│   └── promoManager.js
├── events/
│   └── eventBindings.js
└── utils/
    ├── modal.js
    ├── formatters.js
    └── domHelpers.js
```

Module Responsibilities

`admin.js`

Responsibility:

- Bootstrap/orchestration only
- Initialization logic only
- Startup loading
- Initial render trigger
- Role protection
- Minimal glue code

Keep:

```
init()
handleLogout()
```

It must NOT become another giant file.

`state/adminState.js`

Move global state here.

Current globals include:

```
allUsers
allFamilies
allRequests
allInvites
pendingRequests
currentFamily
allServices
currentReviewId
allActivities
allSubscriptions
userFilterRole
userFilterStatus
subFilter
uploadedPromoImageBase64
```

Implement as a shared state object.

Example direction:

```
export const adminState = {
  allUsers: [],
  ...
}
```

All modules must read/write shared state through this object.

No hidden globals.

loaders/dataLoaders.js

Move:

```
loadUsers()  
loadFamilies()  
loadRequests()  
loadInvites()  
loadServices()  
loadSubscriptions()  
loadActivities()
```

Responsibilities:

- API fetching
- state population

No rendering logic.

navigation/viewManager.js

Move:

```
showView()
```

Responsibilities:

- section visibility
- view switching
- render orchestration

Must still trigger the appropriate renderers.

navigation/sidebar.js

Move:

```
toggleSidebar()  
closeSidebar()
```

overview/overviewRenderer.js

Move:

```
renderOverview()
```

Responsibilities:

- dashboard counts
 - alerts
 - overview metrics
 - badges
-

overview/activityFeed.js

Move:

```
loadActivities()  
renderActivitiesFeed()  
formatTimeAgo()
```

Keep tightly coupled.

users/userFilters.js

Move:

```
toggleFilterPanel()  
setFilter()  
clearFilters()  
applyUserFilters()
```

Responsibilities:

- filtering
 - searching
 - filter state
-

users/userRenderer.js

Move:

```
renderUsers()  
renderUsersTableData()
```

Responsibilities:

- users table rendering only
-

users/userActions.js

Move:

```
approveUser()  
changeRole()
```

Responsibilities:

- mutations
- API actions

Must trigger appropriate rerenders after updates.

users/userDetails.js

Move:

```
openUserDetails()
```

Responsibilities:

- user analytics modal
 - engagement score
 - subscriptions rendering
 - request/invite history
-

requests/requestRenderer.js

Move:

renderRequests()

requests/requestReview.js

Move:

openReviewModal()
doReview()

Keep coupled.

families/familyRenderer.js

Move:

renderFamilies()

families/familyModal.js

Move:

openCreateFamilyModal()
updatePlansDropdown()

families/familyMembers.js

Move:

```
openMembersModal()  
removeMemberFromFamily()
```

families/addMemberView.js

Move:

```
showAddMemberView()  
addMemberToFamily()
```

subscriptions/subscriptionRenderer.js

Move:

```
setSubFilter()  
renderSubscriptions()
```

Keep together.

invites/inviteRenderer.js

Move:

```
renderInvites()
```

invites/inviteModal.js

Move:

```
openCreateInviteModal()  
updateInvitePlanInfo()
```

promos/promoManager.js

Move:

```
previewPromoImage()  
publishPromo()  
bringDownPromo()  
loadAndRenderPromos()
```

Keep together.

events/eventBindings.js

Move the entire DOMContentLoaded block here.

Responsibilities:

- DOM event listeners
- binding UI to modules

This module should wire everything together cleanly.

utils/modal.js

Move:

```
closeModal()
```

utils/domHelpers.js

Move:

```
escHtml()
```

utils/formatters.js

Move reusable helpers:

```
formatTimeAgo()
```

Also move shared formatters if locally defined elsewhere:

```
fmtDate()  
roleBadge()  
statusBadge()
```

Only if they exist in the codebase.

Communication Rules

You MUST ensure module communication works correctly.

Required flow:

```
admin.js  
  ↓  
dataLoaders.js  
  ↓  
adminState.js  
  ↓  
renderers read state  
  ↓  
eventBindings trigger actions  
  ↓  
actions mutate state  
  ↓  
rerender UI
```

No circular dependency spaghetti.

No duplicated state.

No stale references.

No direct mutation of unrelated modules.

Import Strategy

Use **ES Modules** (`import` / `export`) where appropriate.

However:

If HTML currently uses inline handlers like:

```
onclick=""  
onchange=""  
onsubmit=""
```

do NOT break them.

Expose only required functions intentionally to `window`.

Example:

```
window.openMembersModal = openMembersModal;
```

Only for functions actually needed by inline HTML.

Avoid dumping everything onto `window`.

Path Updating Requirement

You MUST update:

- `<script>` tags
- relative import paths
- HTML references
- module paths
- script loading order
- event binding locations
- global references

The project must still run after refactor without manual fixing.

Validation Checklist

Before finishing, verify:

- No functionality changed
- No missing imports
- No duplicate state
- No broken UI
- No console errors
- All modals still work
- Overview still updates
- Families still render
- Requests still review
- Invites still send
- Promos still publish/remove
- Add member flow still works
- Filters still work
- Sidebar still works
- DOMContentLoaded bindings still fire
- All paths updated
- Inline handlers still function
- All related files updated to match new module paths

Finally, explain:

1. Every file created
2. What moved into it
3. Why it communicates with other modules
4. What files were updated outside `admin.js`
5. Any global window bindings intentionally preserved